



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Antispam

Curso: Calidad y Pruebas de Software

Docente: Patrick Cuadros Quiroga

Integrantes:

Mamani Cori, Cristhian Carlos

(2023077282)

Jahuira Pilco, Dayan Elvis

(2022075749)

**Tacna – Perú
2026**

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	CM - DJ	PCQ	PCQ	26/04/2026	Versión Original
2.0	CM - DJ	PCQ	PCQ	29/04/2026	Versión 2.0
3.0	CM - DJ	PCQ	PCQ	06/07/2026	Versión Final

Sistema Antispam

Documento de Especificación de Requerimientos de Software

Versión 3.0

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	CM - DJ	PCQ	PCQ	26/04/2026	Versión Original
2.0	CM - DJ	PCQ	PCQ	29/04/2026	Versión 2.0
3.0	CM - DJ	PCQ	PCQ	06/07/2026	Versión Final

ÍNDICE GENERAL

1. Generalidades de la Empresa	4
1.1. Nombre de la Empresa	4
1.2. Vision	4
1.3. Mision	4
1.4. Organigrama	4
2. Visionamiento de la Empresa	5
2.1. Descripción del Problema	5
2.2. Objetivos de Negocios	5
2.3. Objetivos de Diseño	5
2.4. Alcance del proyecto	5
2.5. Viabilidad del Sistema	6
2.6. Información obtenida del Levantamiento de Informacion	6
3. Análisis de Procesos	7
3.1. Diagrama del Proceso Actual – Diagrama de actividades	7
3.2. Diagrama del Proceso Propuesto – Diagrama de actividades Inicial	8
4. Especificación de Requerimientos de Software	8
4.1. Cuadro de Requerimientos funcionales Inicial	8
4.2. Cuadro de Requerimientos No funcionales	8
4.3. Cuadro de Requerimientos funcionales Final	9
4.4. Reglas de Negocio	10
5. Fase de Desarrollo	12
5.1. Perfiles de Usuario	12
5.2. Modelo Conceptual	13
a) Diagrama de Paquetes	13
b) Diagrama de Casos de Uso	14
c) Escenarios de Caso de Uso (narrativa)	14
5.3. Modelo Logico	27
a) Análisis de Objetos	27
b) Diagrama de Actividades con Objetos	28
c) Diagrama de Secuencia	28
d) Diagrama de Clases	28
6. CONCLUSIONES	28
7. RECOMENDACIONES	29
8. BIBLIOGRAFIA	29
9. WEBGRAFÍA	29

Informe de SRS

1. Generalidades de la Empresa

1.1. Nombre de la Empresa

Aegis Filter

1.2. Vision

Ser una herramienta backend de referencia y código abierto, capaz de integrarse en cualquier plataforma web moderna para proporcionar entornos digitales seguros, limpios y libres de contenido malicioso.

1.3. Mision

Proveer un servicio de filtrado automatizado altamente eficiente y fácil de desplegar mediante contenedores, que reduzca la carga de trabajo manual de los moderadores web y proteja a los usuarios finales de enlaces fraudulentos.

1.4. Organigrama

- Líder de Proyecto / DevOps

Encargado de Terraform y Azure

- Desarrollador Backend

Encargado de Laravel y Motor de Reglas

- Analista QA y Base de Datos

Encargado de Pruebas, Docker y MySQL

2. Visionamiento de la Empresa

2.1. Descripción del Problema

Las secciones de comentarios y foros en las aplicaciones web son vulnerables a ataques masivos de bots que publican enlaces de phishing, publicidad engañosa y contenido irrelevante (Spam). Las soluciones actuales suelen requerir horas de revisión manual, lo que retrasa la publicación de contenido legítimo y expone la base de datos a información indeseada.

2.2. Objetivos de Negocios

- Automatizar la moderación de contenido para reducir el tiempo de revisión manual en un 80%.
- Proporcionar una capa de seguridad preventiva que bloquee los comentarios antes de que sean persistidos en la base de datos de producción.

2.3. Objetivos de Diseño

- Construir una arquitectura basada en microservicios (Backend separado) para que sea invisible al usuario final.
- Garantizar un despliegue ágil, inmutable y escalable utilizando Docker y automatización con GitHub Actions.

2.4. Alcance del proyecto

El sistema interceptará las peticiones HTTP de creación de comentarios (desarrolladas en Laravel), evaluará el texto utilizando un motor de Expresiones Regulares y Listas Negras (SpamFilterService), y

determinará si el registro se inserta en la base de datos MySQL 8 o es rechazado. Todo el entorno estará alojado en la nube de Microsoft Azure.

2.5. Viabilidad del Sistema

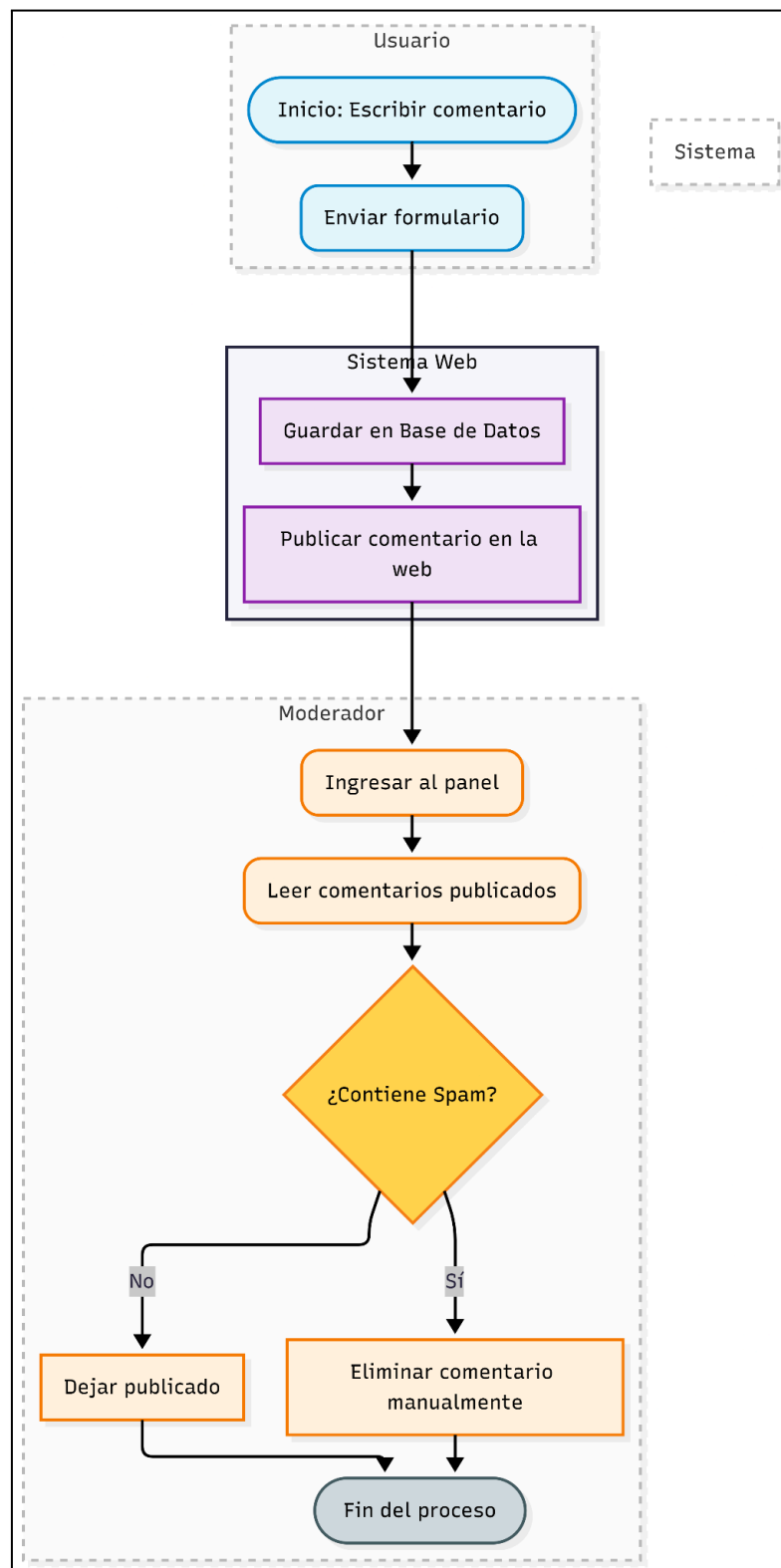
El sistema es factible técnica y económicamente, respaldado por el uso de herramientas Open Source (PHP, Laravel, Debian) e Infraestructura como Código (Terraform) que optimiza el consumo de recursos en la nube.

2.6. Información obtenida del Levantamiento de Información

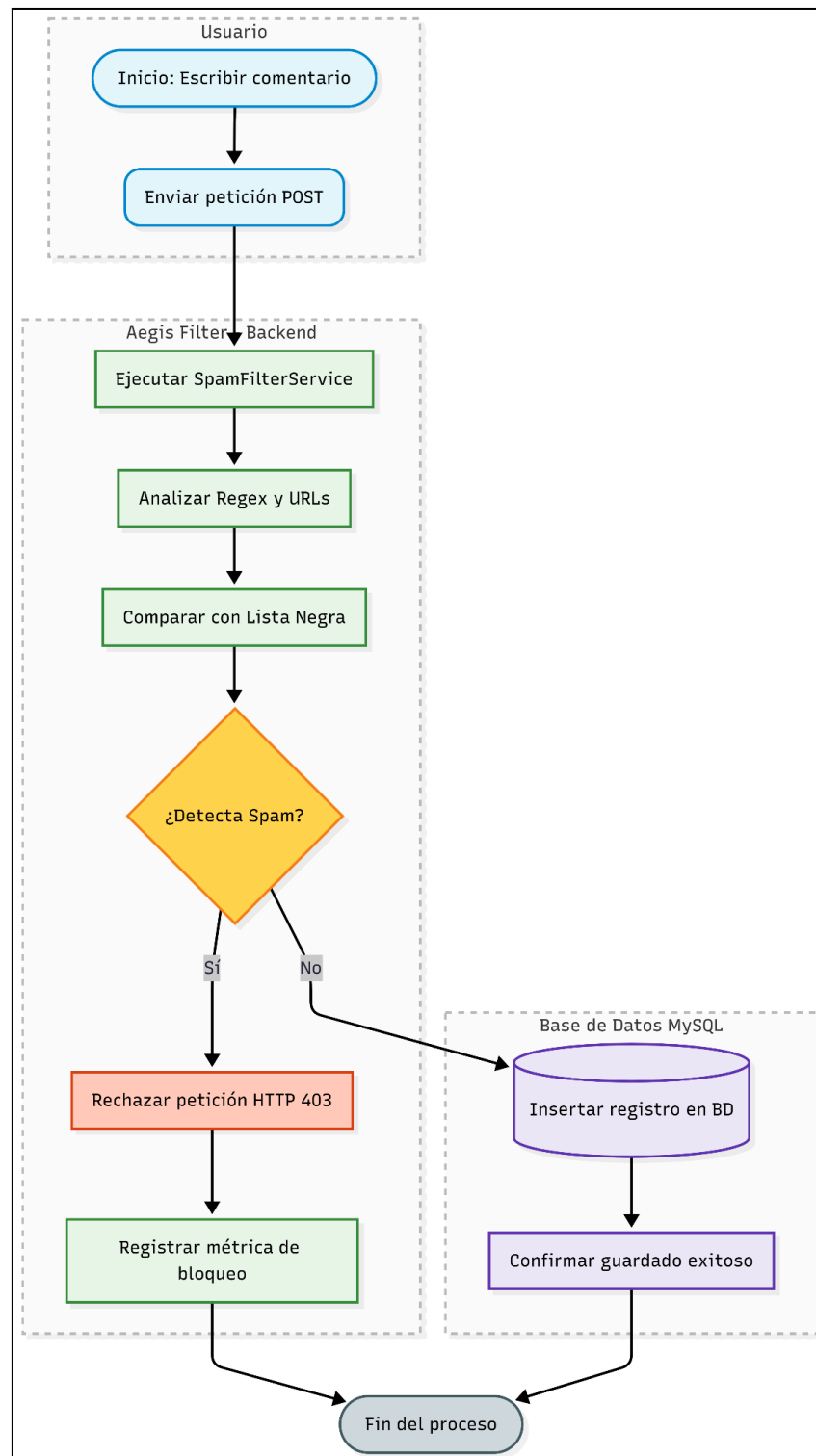
Se determinó que la principal amenaza de spam consiste en textos repetitivos que contienen más de dos enlaces externos o que utilizan palabras clave fraudulentas recurrentes. Por lo tanto, el sistema debe enfocarse en filtrar estos dos vectores principales.

3. Análisis de Procesos

3.1. Diagrama del Proceso Actual – Diagrama de actividades



3.2. Diagrama del Proceso Propuesto – Diagrama de actividades Inicial



4. Especificación de Requerimientos de Software

4.1. Cuadro de Requerimientos funcionales Inicial

ID	Descripción	Prioridad
RF-01	El sistema debe interceptar las peticiones de creación de comentarios en tiempo real.	Muy Alta
RF-02	El sistema debe validar el texto mediante un motor de expresiones regulares para detectar patrones de URL.	Muy Alta
RF-03	El sistema debe comparar el contenido contra una lista negra de términos prohibidos.	Muy Alta
RF-04	El sistema debe registrar métricas sobre la cantidad de comentarios bloqueados y permitidos.	Muy Alta

4.2. Cuadro de Requerimientos No funcionales

ID	Descripción	Prioridad
RNF-01	El sistema debe estar desplegado en Microsoft Azure para garantizar un acceso constante vía web	Muy Alta
RNF-02	La infraestructura debe ser gestionada mediante Terraform (IaC) para asegurar configuraciones de red cerradas (puerto 3306 bloqueado al exterior)	Alta
RNF-03	La aplicación debe estar contenedorizada en Docker para facilitar actualizaciones inmutables	Alta

4.3. Cuadro de Requerimientos funcionales Final

ID	Descripción	Prioridad
----	-------------	-----------

RF-01	El sistema debe interceptar las peticiones de creación de comentarios en tiempo real.	Muy Alta
RF-02	El sistema debe validar el texto mediante un motor de expresiones regulares para detectar patrones de URL.	Muy Alta
RF-03	El sistema debe comparar el contenido contra una lista negra de términos prohibidos.	Muy Alta
RF-04	El sistema debe registrar métricas sobre la cantidad de comentarios bloqueados y permitidos.	Muy Alta

4.4. Reglas de Negocio

ID	Descripción	Prioridad
RNF-01	Un comentario será bloqueado automáticamente si contiene 3 o más enlaces (http o https).	Muy Alta
RNF-02	Si se detecta Spam, el sistema debe retornar un código de estado HTTP 403 (Prohibido) o 422 (Entidad no procesable) para evitar que el registro llegue a la base de datos.	Muy Alta

5. Fase de Desarrollo

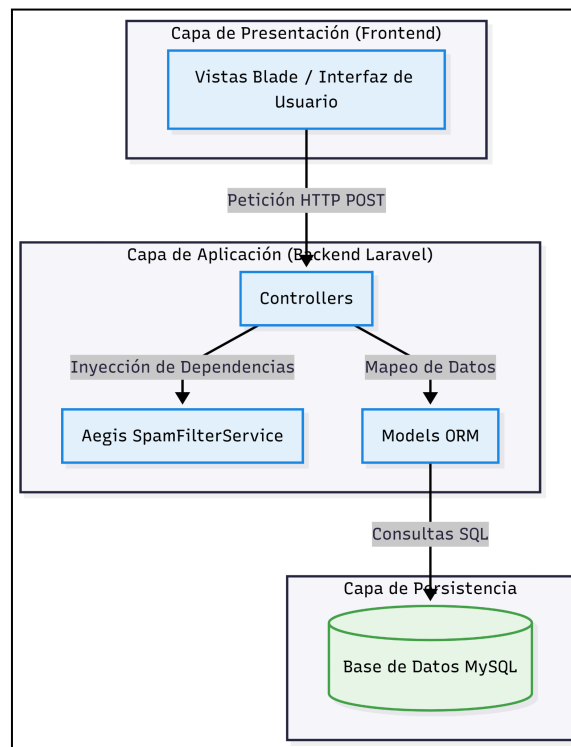
5.1. Perfiles de Usuario

Administrador / DevOps: Encargado del despliegue en Azure y la configuración de reglas en el archivo main.tf.

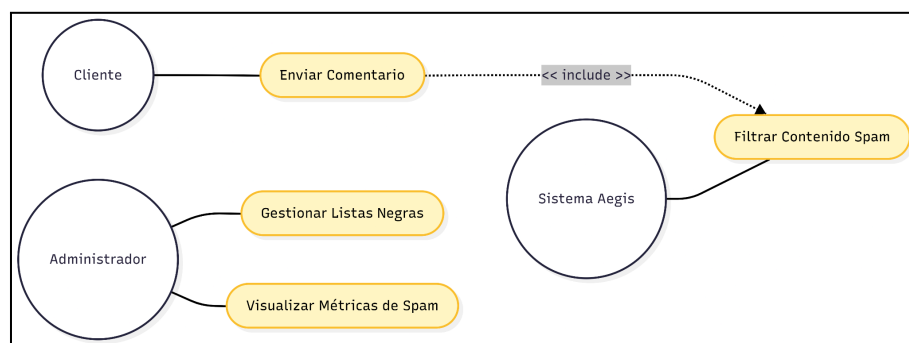
Moderador: Responsable de verificar las estadísticas de filtrado en el panel administrativo.

5.2. Modelo Conceptual

a) Diagrama de Paquetes



b) Diagrama de Casos de Uso



c) Escenarios de Caso de Uso (narrativa)

- Como desarrollador backend...
- Quiero crear un servicio de validación por expresiones regulares...

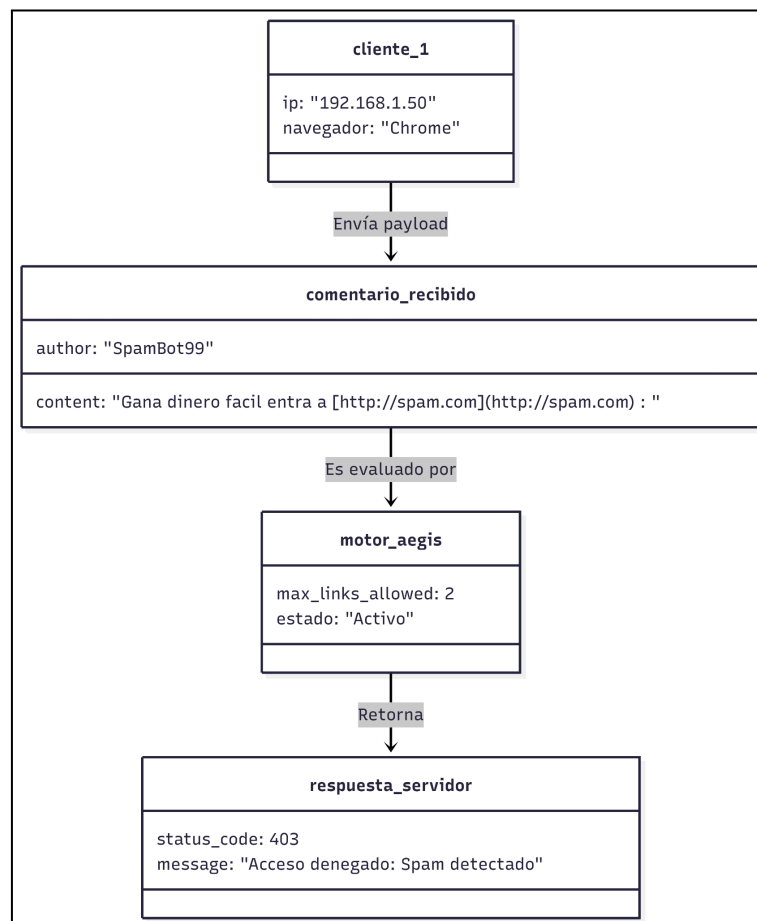
- Para identificar comentarios que contengan múltiples enlaces maliciosos de forma automática.

Escenario Gherkin: Detección exitosa de Spam por URLs.

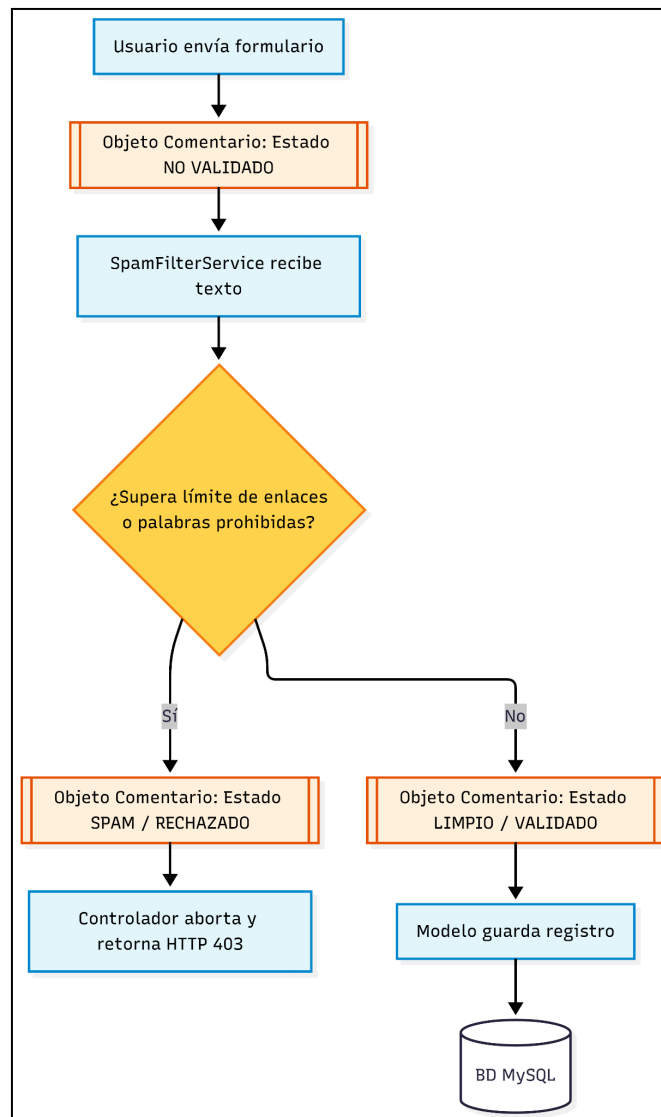
DADO que la regla de negocio limita a 2 los enlaces permitidos,
CUANDO un usuario envía un comentario con 3 direcciones web
diferentes, ENTONCES el sistema debe denegar el acceso a la base de
datos Y registrar el evento como una amenaza bloqueada.

5.3. Modelo Logico

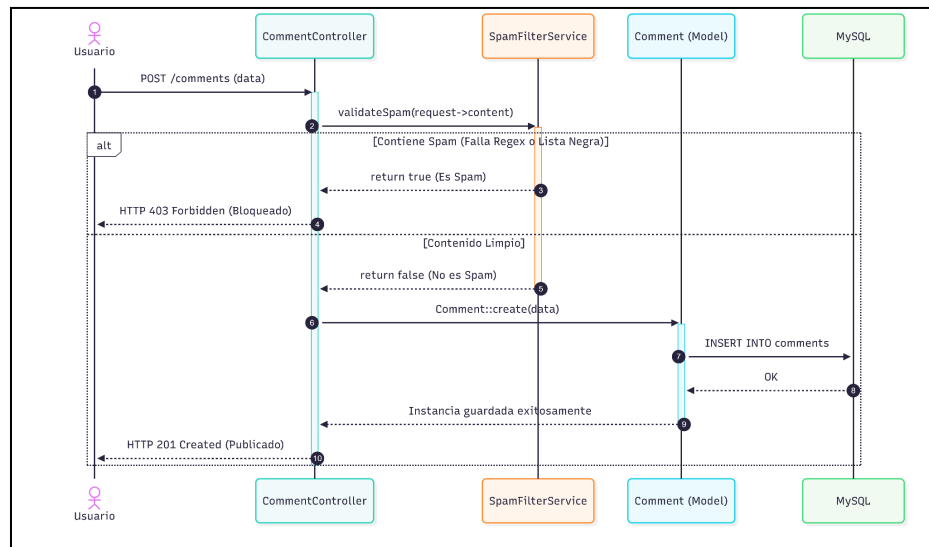
a) *Análisis de Objetos*



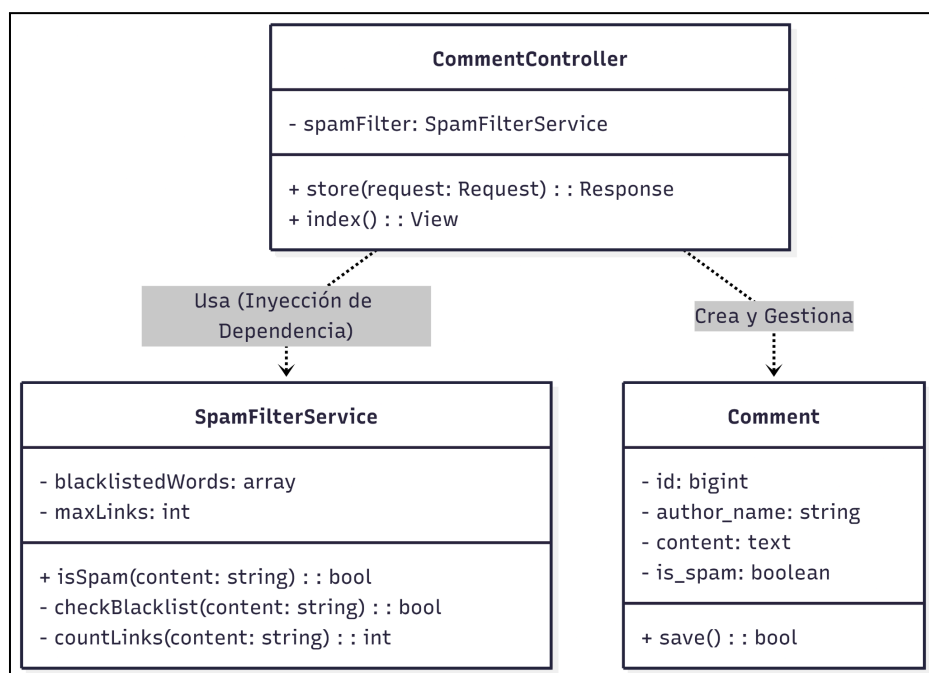
b) Diagrama de Actividades con Objetos



c) Diagrama de Secuencia



d) Diagrama de Clases



6. CONCLUSIONES

El diseño de Aegis Filter bajo la especificación FD03 asegura un filtrado robusto y preventivo. El uso de criterios de aceptación Gherkin

facilitará la creación de pruebas unitarias automáticas en el pipeline de GitHub Actions, garantizando la calidad antes del despliegue.

7. RECOMENDACIONES

Se recomienda integrar las métricas de filtrado con un servicio de alertas (como Telegram o Email) para notificar al administrador en caso de ataques masivos de bots en tiempo real.

8. BIBLIOGRAFIA

Microsoft Azure. (2026). Documentación oficial de máquinas virtuales e infraestructura de Azure.

HashiCorp. (2026). Terraform Registry: Azure Provider Documentation.

Laravel. (2026). Laravel 11.x Documentation: Testing and Services.

Docker Inc. (2026). Docker Compose reference.

9. WEBGRAFIA

Guías de GitHub Actions y Wikis: <https://docs.github.com/es>

Documentación de calidad de código en SonarCloud:

<https://docs.sonarsource.com/sonarcloud/>

Auditoría de seguridad con Snyk: <https://docs.snyk.io/>